

[CS230] Convolutional Neural Networks: Object Detection & Sliding Windows

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-8. Deep Learning Strategy & Architecture - *Completed*

Chapter 9. Convolutional Neural Networks (Current Part)

- 9.1-9.5 CNN Basics, Classic Nets, ResNet, Inception - *Completed*
- **9.6 Object Detection Introduction**
 - * Classification vs Detection
 - * Sliding Windows Algorithm (The Old Way)
 - * **Convolutional Implementation (The Fast Way)**
- 9.7 YOLO Algorithm (You Only Look Once) - *Upcoming*

지난 시간 복습 및 연결

우리는 지금까지 "이 이미지가 고양이인가?"를 맞추는 **분류(Classification)** 문제를 풀었습니다. 하지만 자율주행차라면 어떨까요? "전방에 차가 있다"는 것만으로는 부족합니다. "전방 50m 왼쪽 차선에 있다"는 위치 정보가 필요하며, 동시에 보행자, 신호등도 찾아야 합니다. 이것이 **객체 탐지(Object Detection)**입니다. 오늘은 그 시초인 슬라이딩 윈도우 알고리즘과, 이를 획기적으로 가속화한 합성곱 구현법을 배웁니다.

1 Unit Overview

□ 핵심 목표

이 단원은 객체 탐지의 기본 개념과 속도 문제를 해결하는 핵심 기술을 다룹니다.

- **개념:** 분류(Classification)와 탐지(Detection)의 차이를 이해합니다.
- **고전:** 윈도우를 이동시키며 찾는 슬라이딩 윈도우 방식의 한계를 파악합니다.
- **혁신:** FC 층을 Conv 층으로 변환하여, **단 한 번의 연산**으로 모든 윈도우를 처리하는 기술을 익힙니다.

2 Essential Terminology

용어	질문	출력 예시
Classification	무엇인가?	Cat (1)
Localization	무엇이고 어디에 있는가? (단일 객체)	Cat, b_x, b_y, b_h, b_w
Detection	무엇들이 각각 어디에 있는가? (다중 객체)	Cat(x,y..), Dog(x,y..)

3 Core Concepts: 찾을 때까지 뒤효다

3.1 1. Sliding Windows Algorithm (Basic)

가장 원시적인 방법입니다. 1. 이미지 왼쪽 상단부터 작은 윈도우를 잘라냅니다. 2. 잘라낸 이미지를 ConvNet에 넣어 예측합니다. 3. 옆으로 한 칸 이동(Stride)해서 반복합니다. 4. 다 끝나면 윈도우 크기를 키워서 다시 처음부터 합니다.

치명적 단점: 속도 (오해 방지 가이드)

이 방식은 계산 비용이 폭발합니다. 작은 스트라이드 → 수만 번 ConvNet 실행 → 너무 느림. 큰 스트라이드 → 듬성듬성 봄 → 정확도 하락.

4 Deep Dive: Convolutional Implementation

”어떻게 하면 for-loop 없이 한 번에 처리할까?” 이 섹션이 오늘 강의의 하이라이트입니다.

4.1 1. Turning FC into Conv layers

전통적인 CNN의 마지막은 평탄화(Flatten) 후 FC 층이었습니다. 이를 1×1 Conv 층으로 바꿉니다.

- 기존: $5 \times 5 \times 16 \xrightarrow{\text{Flatten}} 400 \xrightarrow{\text{FC}} 400$
- 변환: $5 \times 5 \times 16 \xrightarrow{\text{Conv}(5 \times 5, 400)} 1 \times 1 \times 400$

수학적으로 값은 완벽히 동일하지만, 이제는 공간적 위치 정보를 유지할 수 있게 되었습니다.

4.2 2. Running on the Whole Image

이제 이미지를 자르지 않고 통째로 넣습니다.

□ 연산 공유의 마법 (수학적 원리)

학습 모델 입력이 14×14 라고 가정합니다. 테스트 때 16×16 이미지를 통째로 넣으면 어떻게 될까요?

- 기존: 4번 잘라서 4번 실행.
- Conv 방식: 16×16 이미지가 네트워크를 통과하면, 최종 출력이 2×2 크기로 나옵니다.
- 해석: (0,0)은 좌상단 윈도우 결과, (0,1)은 우상단 윈도우 결과입니다.
- 결과: 공통 영역의 연산을 공유하므로 속도가 수십 배 빨라집니다.

5 Implementation: Fully Convolutional Network

Keras를 이용해 FC 층을 Conv 층으로 대체하는 모델을 만듭니다.

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3
4 def create_fcn_model(input_shape, num_classes):
5     inputs = layers.Input(shape=input_shape)
6
7     # Feature Extractor
8     x = layers.Conv2D(16, (5, 5), activation='relu')(inputs)
9     x = layers.MaxPooling2D((2, 2), strides=2)(x)
10    x = layers.Conv2D(32, (5, 5), activation='relu')(x)
11    x = layers.MaxPooling2D((2, 2), strides=2)(x)
12
13    # --- 핵심 변환 부분 ---
14    # Flatten 대신 Conv2D 사용
15    # 마지막 특성맵 크기가 5라고 가정할 때, 5x5 커널 사용
16    x = layers.Conv2D(256, (5, 5), activation='relu')(x)
17
18    # 마지막 분류기 (1x1 Conv)
19    outputs = layers.Conv2D(num_classes, (1, 1), activation='softmax')(x)
20
21    model = models.Model(inputs, outputs)
22    return model
23
24 # --- 실행 ---
25 if __name__ == "__main__":
26     # 1. 학습용 작은( 이미지)
27     train_model = create_fcn_model((28, 28, 3), 4)
28     print(train_model.output_shape) # (None, 1, 1, 4)
29
30     # 2. 테스트용 큰( 이미지 통째로 입력)
31     test_input = tf.random.normal((1, 32, 32, 3))
32     test_output = train_model(test_input)
33     print(test_output.shape) # (1, 2, 2, 4) -> 개4 윈도우 결과 동시 출력
```

Listing 1: Fully Convolutional Model

6 FAQ & Pitfalls

Q. 이렇게 하면 바운딩 박스 위치가 정확한가요?

A. 아니요. 윈도우가 고정된 간격(Stride)으로만 움직이기 때문에, 객체가 윈도우 사이에 걸쳐

있거나 크기가 안 맞으면 정확히 잡아내지 못합니다. 이 문제를 해결하기 위해 다음 시간에 배울 **YOLO**가 필요합니다.

Q. 입력 이미지 크기가 계속 바뀌어도 되나요?

A. 네, Fully Convolutional Network는 고정된 크기의 FC 층이 없으므로 입력 크기에 제한이 없습니다. 입력이 커지면 출력 맵($H \times W$)도 커질 뿐입니다.

다음 단계 (Next Step)

우리는 슬라이딩 윈도우를 빠르게 만드는 법을 배웠습니다. 하지만 여전히 ”정확한 박스 위치”를 잡지 못하는 한계가 있습니다.

객체가 어디에 있는 정확하게 박스를 쳐주고(Regression), 심지어 하나의 셀에서 여러 객체를 동시에 찾아내는 실시간 탐지의 끝판왕. 다음 시간에는 [**YOLO (You Only Look Once)**] 알고리즘을 통해 **IOU**와 **Non-max Suppression**의 개념을 정복하겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Detection:** 무엇이(What) 어디에(Where) 있는지 찾는 문제.
2. **Sliding Window:** 윈도우를 이동하며 찾음. 너무 느림.
3. **Conv Implementation:** FC 층을 Conv 층으로 바꾸면 연산을 공유할 수 있다.
4. **Result:** 큰 이미지를 한 번만 통과시키면(One pass) 모든 윈도우 결과가 나온다.